Azure Data Engineering - Consolidated Technical Notes

Azure Data Factory (ADF) | Azure Databricks | PySpark execution | Microsoft Fabric | Monitoring and KQL

1. Azure Data Factory (ADF) activities

• Copy Activity: Moves data between supported sources and sinks, optionally mapping columns and converting formats.

• Lookup: Reads configuration rows, watermark values or control data to drive a pipeline.

• Get Metadata: Retrieves file existence, size, last modified time, child items and other dataset metadata.

• ForEach: Iterates over an array, such as source tables returned by Lookup.

• If Condition / Switch: Branches based on expressions or parameter values.

• Until / Wait: Repeats until a condition is satisfied, or pauses execution.

• Execute Pipeline: Calls a child pipeline with parameters; useful for orchestration.

• Databricks Notebook: Runs a notebook for Bronze, Silver or Gold transformations.

• Web / Webhook / Azure Function: Calls HTTP endpoints, supports callbacks, or invokes functions.

• Stored Procedure / Script / Data Flow: Executes database logic or transformations.

2. Metadata-driven pipeline

Store source definitions in a control table: source system, object name, file path, load type, watermark column, target table, active flag and run order. A Lookup reads active rows; ForEach calls a parameterized ingestion pipeline. Each iteration copies data, records counts and status, and updates the watermark only after a successful commit.

Example flow: Schedule trigger -> Lookup configuration -> ForEach source -> Copy to ADLS Bronze -> Databricks notebook(s) -> audit log -> alert on failure.

3. Integration Runtime (IR) types

IR type	Use case
Azure IR	Cloud-to-cloud movement, data flows, and dispatch of activities in supported networks.
Self-hosted IR (SHIR)	On-premises/private network access; installed on a machine with connectivity to source systems.
Azure-SSIS IR	Runs existing SQL Server Integration Services packages in Azure.

4. Authentication and security

• System-assigned managed identity: Identity attached to one Azure resource; lifecycle follows that resource.

• User-assigned managed identity: Standalone reusable identity that can be attached to multiple resources.

• Service principal: Application identity authenticated with a secret or certificate; prefer managed identity when supported.

• Personal access token (PAT): User-scoped token for supported APIs and tooling, including some Databricks use cases; manage expiration and permissions carefully.

• Best practices: Use least-privilege RBAC/ACLs, Key Vault for secrets, private networking and periodic access reviews.

5. Databricks compute and platform options

Option	Description
Jobs compute	Ephemeral or job-scoped compute for automated ETL and scheduled workloads.
All-purpose compute	Interactive notebooks, development and exploration.
SQL warehouses	SQL execution for analytics, dashboards and BI.
Serverless compute	Databricks-managed infrastructure for eligible workloads.
Classic compute	Compute deployed in the customer cloud account/subscription context.
Photon	Native vectorized engine accelerating supported SQL/DataFrame operations.
Standard / Premium	Workspace pricing tiers; availability and feature differences vary by cloud and offering.

6. PySpark backend execution - step-by-step

• 1. Submit PySpark code in a notebook, job or application.

• 2. Python driver executes Python statements and builds Spark API calls.

• 3. Py4J bridges Python to JVM APIs in classic PySpark execution (implementation varies for Spark Connect).

• 4. SparkSession provides entry to DataFrame and SQL operations.

• 5. Transformations build an unresolved logical plan lazily.

• 6. Catalyst analyzer resolves tables, columns, data types and functions.

• 7. Catalyst optimizer rewrites the analyzed logical plan (e.g., projection/filter pushdown).

• 8. Physical planner chooses execution operators, join strategies and exchanges.

• 9. On an action (show, count, collect, write), Spark executes the plan.

• 10. The execution graph is split into jobs and stages at shuffle boundaries.

• 11. Stages are divided into tasks, typically one task per partition.

• 12. Executors run tasks in parallel; shuffle data is exchanged as required.

• 13. Results are written to storage or returned to the driver for display.

Important distinction: A DAG is derived from dependencies and execution planning; jobs are typically initiated by actions, stages are separated by shuffle boundaries, and tasks operate on partitions. Adaptive Query Execution may revise physical decisions at runtime.

7. DBU, DIU and CU - what they measure

Unit	Platform	Meaning
DBU	Azure Databricks	Databricks Unit: normalized processing consumption used for Databricks service billing; DBU rate varies by workload/compute tier.
DIU	Azure Data Factory	Data Integration Unit: capacity measure for certain Copy Activity executions on Azure IR; does not measure every ADF activity.
CU	Microsoft Fabric	Capacity Unit: standardized Fabric capacity resource; F SKU number indicates provisioned CU quantity.

DBU example: 10 DBU/hour x 2 hours = 20 DBUs. Databricks service cost = consumed DBUs x applicable DBU rate. Classic compute can also incur underlying Azure VM, storage and networking charges. Actual rates depend on region,

SKU and contract.

ADF DIU examples: 4, 8, 16 and 32 DIUs are possible settings for supported Copy Activity scenarios; supported values, effective allocation and billing depend on connector, integration runtime and execution characteristics. DIU is not a general-purpose cluster size.

Fabric capacity examples: F2 = 2 CUs; F4 = 4 CUs; F8 = 8 CUs; F16 = 16 CUs; F32 = 32 CUs; F64 = 64 CUs. Workloads consume capacity over time; throttling/smoothing and licensing details depend on workload and SKU.

8. Observability and Site Reliability Engineering (SRE)

• Azure Monitor: Collects and evaluates platform metrics, logs, alerts and application telemetry.

• Log Analytics workspace: Central log store queried using Kusto Query Language (KQL).

• Application Insights: Application performance monitoring, requests, dependencies, exceptions and traces.

• ADF monitoring: Pipeline/activity run IDs, status, duration, retry counts, copy throughput and failure details.

• Databricks monitoring: Job run logs, Spark UI, executor metrics, SQL query profiles, system tables (where available) and audit logs.

• SRE signals: Latency, traffic, errors and saturation; track SLIs/SLOs and alert on actionable symptoms.

9. KQL - Kusto Query Language

KQL is a read-oriented query language used in Azure Monitor Logs / Log Analytics and Azure Data Explorer. Queries commonly use pipe operators to filter, project, aggregate and sort telemetry. Table availability depends on enabled diagnostic settings and workspace configuration.

• **Filter recent records:** `AzureActivity | where TimeGenerated > ago(24h) | take 20`

• **Summarize errors:** `AppExceptions | where TimeGenerated > ago(1h) | summarize Errors=count() by bin(TimeGenerated, 5m)`

• **Sort latest:** `AzureActivity | project TimeGenerated, OperationNameValue, ActivityStatusValue | order by TimeGenerated desc`

• **Key operators:** where, project, extend, summarize, join, distinct, sort/order by, take, parse, mv-expand.

10. End-to-end reference architecture

Sources (Blob Storage, ADLS Gen2, REST APIs, on-premises databases via SHIR) -> ADF parameterized ingestion -> ADLS Bronze -> Databricks Bronze/Silver/Gold notebooks -> Delta Lake Gold tables -> Power BI / downstream consumers. ADF master pipeline coordinates child pipelines for ingestion, transformations, data quality, logging and notifications.

Production checks: Incremental watermark correctness, idempotent MERGE, schema drift, duplicate handling, quarantine tables, retry/backoff, secret management, RBAC, auditability, data-quality thresholds, and cost monitoring.